Implementierung von Counterfactual Regret Minimization in zunehmend komplexen Pokervarianten

vorgelegt von

Friedemann Doll

EDV.Nr.:924315

dem Fachbereich VI – Informatik und Medien
der Berliner Hochschule für Technik Berlin
vorgelegte Bachelorarbeit
zur Erlangung des akademischen Grades

Bachelor of Engineering (B.Eng.)

im Studiengang

Informatik

Tag der Abgabe 10. Dezember 2025

Gutachter
Prof. Dr.-Ing. Stefan Edlich Berliner Hochschule für Technik

BHT Berliner Hochschule für Technik

Studiere Zukunft

Inhaltsverzeichnis

1 Einleitung **7**

1.1 Motivation und Relevanz 7

1.2 Zielsetzung und Fragestellungen 7

1.3 Aufbau der Arbeit 7

2 Theoretische Grundlagen **9**

2.1 Grundlagen der Spieltheorie 9

2.1.1 Definition und Konzepte 9

2.1.2 Extensive Games with Imperfect Information 11

2.2 Counter Factual Regret Minimization 13

2.2.1 Definitionen 13

2.2.2 Regret Minimization 14

2.2.3 Counterfactual Regret 15

2.2.4 Regret Matching 16

2.2.5 CFR Algorithmus 16

2.3 MCCFR 17

2.3.1 Motivation und Framework 17

2.3.2 Monte Carlo CFR 17

2.3.3 Outcome Sampling MCCFR 18

2.3.4 External Sampling MCCFR 18

2.3.5 Theoretische Eigenschaften 18

2.4 CFR+ 20

2.4.1 Unterschiede zu Vanilla CFR 20

2.4.2 Regret Matching+ 21

2.4.3 Tracking Regret 21

2.4.4 Linear Weighted Average . . . 22
2.5 Hold'em Games . . . 22
2.5.1 Kuhn Poker . . . 24
2.5.2 Leduc Hold'em . . . 24
2.5.3 Zwölf Karten Variante . . . 24
2.5.4 Rhode Island Hold'em . . . 25
2.5.5 Limit Texas Hold'em . . . 25

3 Analyse 27
3.1 Spielvarianten . . . 27
3.2 CFR Solver . . . 28
3.3 Evaluationsmethoden . . . 28

4 Design und Architektur 29
4.1 Game Environment . . . 29
4.2 CFR Solver . . . 31
4.2.1 CFR Training Ablauf . . . 32
4.2.2 Traverse Game Tree . . . 33
4.2.3 Strategy Agent und Logger . . . 33

5 Implementierung 35
5.1 Game Environment . . . 35
5.2 CFR Solver . . . 35
5.3 Evaluationsmethoden . . . 35

6 Evaluation 37
6.1 Ergebnisse . . . 37
6.2 Vergleich der Algorithmen . . . 37
6.3 Vergleich mit Literaturwerten und anderen Modellen . . . 37

7 Fazit 39
7.1 Zusammenfassung . . . 39
7.2 Lessons Learned . . . 39
7.3 Future Work . . . 39

Literaturverzeichnis **40**

Kapitel 1

Einleitung

1.1 Motivation und Relevanz

1.2 Zielsetzung und Fragestellungen

1.3 Aufbau der Arbeit

Kapitel 2

Theoretische Grundlagen

2.1 Grundlagen der Spieltheorie

In diesem Kapitel gebe ich einen kurzen Überblick darüber, was Spieltheorie ist und wo sie angewendet wird. Dabei definiere ich, was ein Spiel und eine Lösung ist. Anschließend umreiße ich die wichtigsten Typen von Spielen und gehe dann genauer auf den Typ der extensiven Spiele mit imperfekter Information ein.

2.1.1 Definition und Konzepte

Die Spieltheorie beschäftigt sich mit der Analyse strategischer Interaktionen zwischen Entscheidungsträgern und bietet hierfür eine Vielzahl analytischer Methoden. Diese basieren auf zwei zentralen Annahmen. Die erste Annahme ist, dass Entscheidungsträger rational handeln.

Ein rationaler Entscheidungsträger kennt seine Handlungsmöglichkeiten, kann Erwartungen über unbekannte Situationen bilden und hat klare Vorstellungen darüber, welche Ergebnisse er bevorzugt. Liegt keine Unsicherheit vor, wählt er die Aktion, die zu den für ihn besten Konsequenzen führt. Diese Präferenzen lassen sich durch eine Nutzenfunktion beschreiben, die jedem möglichen Ergebnis einen Wert zuordnet. Die beste Aktion ist die, welche den höchsten Nutzen erzielt. Sind die Konsequenzen von Aktionen unsicher, maximiert ein rationaler Entscheidungsträger den erwarteten Nutzen. Hierfür muss er in der Lage sein, Wahrscheinlichkeiten für unsichere Ereignisse abzuschätzen [Osborne and Rubinstein, 1994, Kap. 1.4].

Die zweite Annahme ist Entscheidungsträger nutzen ihr Wissen oder ihre Erwartungen über das Verhalten anderer Spieler bei ihrer Entscheidungsfindung. Dies nennt man auch strategisches Handeln. Ein anschauliches Beispiel hierfür ist das Gefangenendilemma. Zwei Verdächtige werden getrennt verhört. Jeder muss entscheiden, ob er schweigt oder den anderen belastet. Die optimale Entscheidung hängt davon ab, was der andere Gefangene vermutlich tut. Strategisches Handeln bedeutet, solche Erwartungen über das Verhalten anderer Spieler in die Entscheidungsfindung einzubeziehen.

Spieltheoretische Modelle befinden sich auf einer hohen Abstraktionsebene und können daher verwendet werden, um ein breites Spektrum an Phänomenen zu betrachten. Zur

formalen Darstellung dieser Modelle werden mathematische Definitionen genutzt. Die behandelten Konzepte sind jedoch nicht grundlegend mathematisch und können auch ohne mathematische Notation beschrieben werden.

Die Spieltheorie findet Anwendung in unzähligen Bereichen. Zu den klassischen Anwendungsgebieten zählen die Analyse wirtschaftlicher und politischer Wettbewerbssituationen. Darüber hinaus wird sie auch in der Evolutionsbiologie, der Soziologie und anderen Disziplinen genutzt, um strategische Interaktionen zu modellieren.[Osborne and Rubinstein, 1994 Kap. 1.1]

Ein Spiel beschreibt eine strategische Interaktion zwischen Spielern mit festgelegten Rahmenbedingungen. Es definiert, welche Aktionen den Spielern zur Verfügung stehen und welche Ergebnisse in ihrem Interesse sind. Das Spiel legt damit die Struktur und die Regeln der Interaktion fest, bestimmt aber nicht, wie sich die Spieler tatsächlich verhalten werden.

Die Frage, welche Aktionen die Spieler tatsächlich ausführen, wird nicht durch das Spiel selbst beantwortet, sondern durch die Lösung des Spiels. Eine Lösung ist ein Konzept, das systematisch beschreibt, welche Ergebnisse in einer Klasse von Spielen unter bestimmten Verhaltensannahmen zu erwarten sind. Sie liefert damit eine Vorhersage über das tatsächliche Verhalten der Spieler unter den getroffenen Annahmen über Rationalität und strategisches Denken.

Spiele können anhand verschiedener Kriterien systematisch kategorisiert werden.

Grundlegend für alle spieltheoretischen Modelle ist das Konzept des Spielers. Ein Spieler kann dabei sowohl eine einzelne Person als auch eine Gruppe von Personen darstellen, die gemeinsam eine Entscheidung treffen.

Die erste grundlegende Unterscheidung betrifft kooperative und nicht kooperative Spiele. Nicht kooperative Spiele modellieren die Aktionen einzelner Spieler. Kooperative Spiele hingegen konzentrieren sich darauf, was Gruppen von Spielern gemeinsam erreichen können. Dabei wird nicht betrachtet wie diese Gruppen intern funktionieren.

Eine weitere wichtige Kategorisierung betrifft die Darstellungsform des Spiels. Hierbei werden strategische Spiele, auch als Spiele in Normalform bezeichnet, von extensiven Spielen unterschieden. Bei strategischen Spielen wählen die Spieler ihre Strategie zu Beginn des Spiels und passen diese während des Spielverlaufs nicht mehr an. Die Entscheidungen werden zudem simultan getroffen, sodass keine Informationen über die Aktionen der Gegenspieler verfügbar sind. Extensive Spiele hingegen spezifizieren die möglichen Reihenfolgen von Ereignissen und erlauben es jedem Spieler, seinen Aktionsplan nicht nur zu Beginn, sondern auch bei jeder weiteren Entscheidung zu überdenken.

Die letzte hier zu nennende Unterscheidung betrifft den Informationsstand der Spieler. Bei Spielen mit perfekter Information sind alle Teilnehmer vollständig über die Züge der anderen Spieler informiert. Bei Spielen mit imperfekter Information hingegen können die Spieler unvollständig über die Aktionen der anderen oder über Ausgänge von Zufallsereignissen informiert sein. [Osborne and Rubinstein, 1994, Kap. 1.2]

2.1.2 Extensive Games with Imperfect Information

Pokerspiele sind extensive Spiele mit imperfekter Information. Da diese Arbeit CFR in Pokervarianten implementiert, wird dieser Spieltyp im Folgenden detailliert betrachtet.

Definition

Ein extensives Spiel mit imperfekter Information modelliert eine sequenzielle Interaktion zwischen Spielern, bei der nicht alle Spieler stets vollständig über alle vergangenen Aktionen informiert sind. Dabei werden die möglichen Spielverläufe als Historien erfasst, wobei eine Historie eine Abfolge von Aktionen darstellt. Zu jedem Zeitpunkt ist entweder ein Spieler oder der Zufall am Zug. Um die imperfekte Information darzustellen, werden Informationssets verwendet. Diese fassen Zustände zusammen, die ein Spieler nicht voneinander unterscheiden kann. Dies kann sowohl vergangene Aktionen anderer Spieler als auch Ausgänge von Zufallsereignissen betreffen.

Ein extensives Spiel mit imperfekter Information lässt sich formal durch folgende Komponenten definieren:

- Eine Menge H von Historien. Eine Historie ist eine Sequenz (endlich oder unendlich) von Aktionen, getätigt von Spielern oder dem Zufall. Die Menge H erfüllt folgende Eigenschaften:
  - Die leere Sequenz $\emptyset$ ist ein Element von H.
  - Ist $(a^k)_{k=1,\dots,K} \in H$ (wobei K unendlich sein kann) und $L < K$, dann gilt $(a^k)_{k=1,\dots,L} \in H$. Das bedeutet, wenn eine Sequenz in H ist, dann ist auch jeder Präfix dieser Sequenz in H.
  - Wenn für eine unendliche Folge $(a^k)_{k=1}^{\infty}$ die Bedingung $(a^k)_{k=1,\dots,L} \in H$ für jede positive ganze Zahl L erfüllt ist, dann gilt $(a^k)_{k=1}^{\infty} \in H$.
- Eine Historie $(a^k)_{k=1,\dots,K} \in H$ ist terminal, wenn sie unendlich ist oder wenn kein a^{K+1} existiert, so dass $(a^k)_{k=1,\dots,K+1} \in H$ gilt. Die Menge aller terminalen Historien wird mit Z bezeichnet. Die Menge der Aktionen, die nach einer nicht-terminalen Historie h verfügbar sind, wird mit $A(h) = \{a : (h, a) \in H\}$ bezeichnet.
- Die Funktion P ordnet jeder nicht-terminalen Historie $h \in H \setminus Z$ einen Spieler aus N oder c zu. $P(h)$ ist der Spieler, der nach h am Zug ist. Ist $P(h) = c$, wählt der Zufall die nächste Aktion.
- Die Funktion f_c ordnet jeder Historie h mit $P(h) = c$ eine Zufallsverteilung $f_c(\cdot \mid h)$ auf $A(h)$ zu. $f_c(a \mid h)$ ist die Wahrscheinlichkeit, dass nach h die Aktion a folgt. Diese Verteilungen sind unabhängig voneinander.
- Für jeden Spieler $i \in N$ werden die Historien h, bei denen $P(h) = i$ gilt, in Gruppen eingeteilt. Diese Menge heißt Informationspartition $\mathcal{I}i$ von Spieler i. Eine solche Gruppe $I_i \in \mathcal{I}_i$ heißt Informationsset von Spieler i. Historien h und h', die zum selben Informationsset I_i gehören, sind für Spieler i nicht unterscheidbar.

Wesentlich ist dabei, dass alle Historien in einem Informationsset I_i die gleichen verfügbaren Aktionen haben, also $A(h) = A(h')$ für alle $h, h' \in I_i$. Für ein Informationsset I_i schreiben wir $A(I_i)$ für die verfügbaren Aktionen und $P(I_i)$ für den Spieler, der am Zug ist.

- Für jeden Spieler $i \in N$ gibt es eine Präferenzrelation $\succeq_i$ über Lotterien auf Z. Eine Lotterie ist eine Wahrscheinlichkeitsverteilung über Endzustände. Es existiert eine Nutzenfunktion $u_i : Z \to \mathbb{R}$, so dass für zwei Lotterien L_1, L_2 gilt: $L_1 \succeq_i L_2 \iff \mathbb{E}[u_i \mid L_1] \geq \mathbb{E}[u_i \mid L_2]$

Strategie

Eine reine Strategie von Spieler $i \in N$ ordnet jedem Informationsset $I \in \mathcal{I}_i$ genau eine zulässige Aktion $\sigma_i(I) \in A(I)$ zu. Sie legt somit einen vollständigen Handlungsplan für alle Informationslagen von i fest. Eine gemischte Strategie α_i ist eine Wahrscheinlichkeitsverteilung über der Menge der reinen Strategien von i. Eine behaviorale Strategie β_i weist für jedes Informationsset $I \in \mathcal{I}_i$ eine Verteilung $\beta_i(\cdot \mid I)$ über $A(I)$ zu. Der Unterschied liegt darin, dass bei einer gemischten Strategie vor Spielbeginn zufällig eine komplette reine Strategie ausgewählt wird, die dann während des gesamten Spiels deterministisch befolgt wird. Bei einer behavioralen Strategie wird hingegen an jedem Informationsset unabhängig eine Wahrscheinlichkeitsverteilung über die verfügbaren Aktionen verwendet, sodass die zufällige Auswahl der Aktionen während des Spiels erfolgt. Unter der Annahme von perfect recall erzeugen beide Repräsentationen dieselben Wahrscheinlichkeitsverteilungen über die Endzustände des Spiels.[1]

Ein Profil behavioraler Strategien $\beta = (\beta_i)_{i \in N}$ zusammen mit den Zufallszügen f_c bestimmt eine Wahrscheinlichkeitsverteilung über die terminalen Historien Z. Daraus lässt sich für jede Auszahlungsfunktion u_i der Erwartungswert $\mathbb{E}[u_i \mid \beta, f_c]$ berechnen. Ein Nash-Gleichgewicht ist ein Strategienprofil, in dem kein Spieler seinen erwarteten Nutzen durch eine einseitige Abweichung seiner Strategie verbessern kann. Formal ist ein Strategienprofil σ^* ein Nash-Gleichgewicht in gemischten Strategien, wenn für jeden Spieler $i \in N$ gilt:

$$O(\sigma^*_{-i}, \sigma^*_i) \succeq_i O(\sigma^*_{-i}, \sigma_i) \tag{2.1}$$

für jede gemischte Strategie σ_i von Spieler i, wobei $O(\sigma)$ die durch σ entstehende Lotterie über Endzuständen ist.

2.2 Counter Factual Regret Minimization

In diesem Kapitel behandle ich die Theorie des Herzstücks dieser Arbeit, des Counterfactual Regret Minimization Algorithmus. Die fachliche Grundlage bildet das Paper „Regret Minimization in Games with Incomplete Information “ [Zinkevich et al., 2007], in welchem der Counterfactual Regret Minimization Algorithmus zum ersten Mal präsentiert wurde. Hierbei beginne ich mit einer Erklärung des Konzepts Regret. Anschließend erkläre ich wann ein Algorithmus als Regret minimierend gilt. Dann erläutere ich, dass von Zinkevich eingeführte Konzept des Counterfactual Regrets und darauffolgend Regret Matching. Zum Ende werden dann die einzelnen Komponenenten sinnvoll zu einem Algorithmus zusammengefügt.

2.2.1 Definitionen

Für ein Strategieprofil σ definieren wir folgende Wahrscheinlichkeiten und Werte:

- $\pi^\sigma(h)$ bezeichnet die Wahrscheinlichkeit, dass History h auftritt, wenn alle Spieler gemäß σ handeln.

[1] [Osborne and Rubinstein, 1994, Ch. 11, Sec. 11.1–11.2]

- $\pi^\sigma(h)$ kann in ein Produkt zerlegt werden: $\pi^\sigma = \prod_{i \in N \cup \{c\}} \pi_i^\sigma(h)$, welches sich zusammensetzt aus den Wahrscheinlichkeiten der einzelnen Spieler und des Zufalls
- $\pi_i^\sigma(h)$ ist die Wahrscheinlichkeit, dass Spieler i in all seinen Entscheidungsknoten auf dem Pfad zu h gemäß σ die Aktionen gewählt hat, die zu h führen.
- $\pi_{-i}^\sigma(h)$ ist das Produkt der Wahrscheinlichkeiten aller anderen Spieler (inklusive Chance), die auf dem Pfad zu h auftreten.
- Für ein Informationsset I ist $\pi^\sigma(I) = \sum_{h \in I} \pi^\sigma(h)$ die Wahrscheinlichkeit, I zu erreichen.
- Der Overall Value $u_i(\sigma)$ gibt den erwarteten Payoff für Spieler i unter Strategieprofil σ an. Er berechnet sich als Summe über alle terminalen Histories h, wobei für jedes Spielende der Payoff $u_i(h)$ mit der Wahrscheinlichkeit $\pi^\sigma(h)$, dass dieses Spielende erreicht wird, multipliziert wird: $u_i(\sigma) = \sum_{h \in Z} u_i(h)\pi^\sigma(h)$.

2.2.2 Regret Minimization

Zunächst definieren wir das Nash Gleichgewicht für ein Zweispieler Nullsummenspiel als Strategienprofil $\sigma = (\sigma_1, \sigma_2)$, welches folgende Ungleichungen erfüllt.

$$u_1(\sigma) \geq \max_{\sigma_1' \in \Sigma_1} u_1(\sigma_1', \sigma_2), \qquad u_2(\sigma) \geq \max_{\sigma_2' \in \Sigma_2} u_2(\sigma_1, \sigma_2') \tag{2.2}$$

([Zinkevich et al., 2007, Eq. 1]). Ein Approximation eines Nash Gleichgewichts, auch ε-Nash Gleichgewicht genannt, wird definiert als:

$$u_1(\sigma) + \varepsilon \geq \max_{\sigma_1' \in \Sigma_1} u_1(\sigma_1', \sigma_2), \qquad u_2(\sigma) + \varepsilon \geq \max_{\sigma_2' \in \Sigma_2} u_2(\sigma_1, \sigma_2') \tag{2.3}$$

([Zinkevich et al., 2007, Eq. 2]).

Das Konzept des Regrets kommt aus dem Online Lernen und hat eine Menge an Lernalgorithmen hervorgebracht. Regret misst den Unterschied zwischen dem maximal erreichbarem Nutzen, unter Verwendung einer optimalen Strategie und dem tatsächlich erzielten Nutzen.

Um Regret Minimierung zu definieren, betrachten wir zunächst das wiederholte Spielen eines extensiven Spiels. Dabei definieren wir folgende Dinge: Sei σ_i^t die von Spieler i in Runde t verwendete Strategie. Der durchschnittliche *overall regret* von Spieler i zum Zeitpunkt T ist

$$R_i^T = \frac{1}{T} \max_{\sigma_i^* \in \Sigma_i} \sum_{t=1}^{T} \big(u_i(\sigma_i^*, \sigma_{-i}^t) - u_i(\sigma^t)\big). \tag{2.4}$$

([Zinkevich et al., 2007, Eq. 3])

Die Durchschnittsstrategie $\bar{\sigma}_i^T$ für Spieler i von Zeit 1 bis T wird für jeden Informationszustand $I \in \mathcal{I}_i$ und jede Aktion $a \in A(I)$ definiert als

$$\bar{\sigma}_i^T(I)(a) = \frac{\sum_{t=1}^{T} \pi_i^{\sigma^t}(I)\, \sigma^t(I)(a)}{\sum_{t=1}^{T} \pi_i^{\sigma^t}(I)}. \tag{2.5}$$

([Zinkevich et al., 2007, Eq. 4]) Die Durchschnittsstrategie gewichtet Aktionen proportional zur Erreichbarkeitswahrscheinlichkeit $\pi_i^{\sigma^t}(I)$ des Informationszustands I in Runde t.

In einem Nullsummenspiel gilt: Ist der durchschnittliche overall regret beider Spieler zum Zeitpunkt T kleiner als ε, dann ist $\bar{\sigma}^T$ ein 2ε-Nash Gleichgewicht ([Zinkevich et al., 2007, Thm. 2]). Ein Algorithmus zum Finden von σ_i^t ist Regret minimierend, wenn der durchschnittliche overall regret von Spieler i sich der Null annähert, während t gegen Unendlich läuft. Aufgrund dieses Zusammenhangs können Regret minimierende Algorithmen im Selfplay verwendet werden, um ein approximatives Nash Gleichgewicht zu berechnen.

2.2.3 Counterfactual Regret

Die zentrale Idee welche Zinkevich et. al. in ihrem Paper "Regret Minimization in Games with Incomplete Information"präsentieren ist, den gesamten Regret Wert in eine Menge aus additiven Termen zu zerlegen. Diese Terme können dann unabhängig voneinander minimiert werden. Insbesondere wird ein ein völlig neues regret Konzept mit dem Namen „Counterfactual Regret “eingeführt, welches für jedes Informationset definiert ist.

Um das Konzept zu erläutern, betrachten wir zunächst ein Informationset $I \in \mathcal{I}_i$. Der counterfactual Nutzen $u_i(\sigma, I)$ ist der erwartete Nutzen unter der Bedingung, dass I erreicht wird und alle Spieler mit σ agieren, während Spieler i so aggiert um den Zustand I zu erreichen. Mit $\pi^\sigma(h, h')$ als Wahrscheinlichkeit, von h nach h' zu gelangen, gilt

$$u_i(\sigma, I) = \frac{\sum_{h \in I,\, h' \in Z} \pi_{-i}^\sigma(h)\, \pi^\sigma(h, h')\, u_i(h')}{\pi_{-i}^\sigma(I)}. \tag{2.6}$$

([Zinkevich et al., 2007, Eq. 5])

Für alle $a \in A(I)$ definieren wir $\sigma|_{I \to a}$ als ein Strategieprofil welches identisch zu σ ist, ausser das Spieler i immer Aktion a ausführt wenn er Informationset I erreicht. Der *immediate counterfactual regret* ist

$$R_{i,\text{imm}}^T(I) = \frac{1}{T} \sum_{t=1}^{T} \max_{a \in A(I)} \pi_{-i}^{\sigma^t}(I) \left(u_i(\sigma^t|_{I \to a}, I) - u_i(\sigma^t, I) \right). \tag{2.7}$$

([Zinkevich et al., 2007, Eq. 6]) Dieser Term misst den Regret im Informationset I, gewichtet mit der counterfaktischen Wahrscheinlichkeit, dass I erreicht werden würde, wenn Spieler i es erzwingen wollte. Da für uns hauptsächlich die positiven regret Werte relevant sind, definieren wir die positiven Immediate Counterfactual Regret Werte als

$$R_{i,\text{imm}}^{T,+}(I) = \max\left(R_{i,\text{imm}}^T(I), 0\right) \tag{2.8}$$

Theorem 3 beschreibt die erste große Erkenntnis des Papers.

$$R_i^T \leq \sum_{I \in \mathcal{I}_i} R_{i,\text{imm}}^{T,+}(I) \tag{2.9}$$

([Zinkevich et al., 2007, Thm. 3])
Das heisst, wenn wir den Immediate Counterfactual Regret minimieren, dann minimieren wir auch den Average Overall Regret.

2.2.4 Regret Matching

Mit folgender Formel berechnen wir den Counterfactual Regret für eine konkrete Aktion a in einem bestimmten Infoset.

$$R_i^T(I,a) = \frac{1}{T}\sum_{t=1}^{T} \pi_{-i}^{\sigma^t}(I)\left(u_i(\sigma^t|_{I\to a}, I) - u_i(\sigma^t, I)\right) \tag{2.10}$$

([Zinkevich et al., 2007, Eq. 7])

Definieren wir $R_i^{T,+}(I,a) = \max(R_i^T(I,a), 0)$ als positiven Anteil des Regrets für Aktion a.

Die Strategie für Iteration $T+1$ wird dann durch Regret Matching bestimmt:

$$\sigma_i^{T+1}(I)(a) = \begin{cases} \frac{R_i^{T,+}(I,a)}{\sum_{a'\in A(I)} R_i^{T,+}(I,a')} & \text{falls } \sum_{a'\in A(I)} R_i^{T,+}(I,a') > 0 \\ \frac{1}{|A(I)|} & \text{in jedem anderen Fall.} \end{cases} \tag{2.11}$$

([Zinkevich et al., 2007, Eq. 8]) Diese Formel weist Aktionen Wahrscheinlichkeiten proportional zu ihrem positiven Regret zu. Gibt es keine positiven Regrets, werden alle Aktionen gleichverteilt gewählt.

Theorem 4 zeigt, dass bei Verwendung von Gleichung 8 der immediate counterfactual regret und der overall regret folgendermaßen beschränkt sind:

$$R_{i,\text{imm}}^T(I) \leq \frac{\Delta_{u,i}\sqrt{|A_i|}}{\sqrt{T}}, \tag{2.12}$$

$$R_i^T \leq \frac{\Delta_{u,i}|\mathcal{I}_i|\sqrt{|A_i|}}{\sqrt{T}}, \tag{2.13}$$

wobei $\Delta_{u,i} = \max_z u_i(z) - \min_z u_i(z)$ der Nutzenbereich und $|A_i| = \max_{h:P(h)=i} |A(h)|$ die maximale Anzahl verfügbarer Aktionen für Spieler i ist ([Zinkevich et al., 2007, Thm. 4]).

2.2.5 CFR Algorithmus

Das CFR Verfahren verbindet die Komponenten aus den vorherigen Abschnitten zu einer wiederholten Selbstspielprozedur. Zwei Spieler spielen ein Spiel T-mal gegeneinander und bestimmen ihre Strategie in jeder Iteration nach Gleichung 8 Für jedes Informationsset I und jede Aktion a speichern wir den kumulierten Regret $R_i^t(I,a)$ und aktualisieren ihn nach jeder Wiederholung. Parallel dazu wird die Durchschnittsstrategie gemäß Gleichung 4 aktualisiert. Nach T Iterationen gibt der Algorithmus $(\bar{\sigma}_1^T, \bar{\sigma}_2^T)$ als approximatives Nash Gleichgewicht zurück.

2.3 MCCFR

2.3.1 Motivation und Framework

Die von Zinkevich et al. präsentierte Vanilla Version von CFR erfordert in jeder Iteration eine vollständige Traversierung des gesamten Game Trees. Dies macht den Algorithmus für sehr große Spiele unpraktikabel.

MCCFR stellt ein allgemeines Framework für Sampling in Counterfactual Regret Minimization dar. Es definiert eine Familie von Monte Carlo CFR Algorithmen, die sich darin unterscheiden, wie sie den Game Tree in jeder Iteration sampeln. Vanilla CFR und eine Verallgemeinerung des chance sampled CFR von Zinkevich et al. sind Spezialfälle dieses Frameworks. Zusätzlich werden zwei neue Mitglieder dieser Familie eingeführt: Outcome Sampling, bei dem nur ein einzelnes Spiel pro Iteration gesampelt wird, und External Sampling, das Chance Knoten und Aktionen des Gegners sampelt.

Unter einer angemessenen Sampling Strategie minimiert jedes Mitglied dieser Familie den overall regret und kann daher für die Berechnung von Nash Gleichgewichten verwendet werden.

2.3.2 Monte Carlo CFR

Der Ansatz reduziert die Kosten pro Iteration, indem er den Game Tree nicht vollständig traversiert und dennoch unverzerrte immediate counterfactual regrets erhält. Dazu werden die in jeder Iteration betrachteten terminal histories eingeschränkt.

Sei $Q = \{Q_1, \ldots, Q_r\}$ eine Menge von Teilmengen von Z, deren Vereinigung die Menge Z umfasst. Jede dieser Teilmengen Q_j wird als Block bezeichnet. In jeder Iteration wird einer dieser Blöcke gesampelt und nur die terminal histories in diesem Block werden betrachtet. Sei $q_j > 0$ die Wahrscheinlichkeit, Block Q_j in der aktuellen Iteration zu betrachten, wobei $\sum_{j=1}^{r} q_j = 1$ gilt. Die Wahrscheinlichkeit, eine terminal history z in der aktuellen Iteration zu betrachten, ist $q(z) = \sum_{j:z \in Q_j} q_j$.

Der sampled counterfactual value beim Aktualisieren von Block j ist:

$$\tilde{v}_i(\sigma, I|j) = \sum_{z \in Q_j \cap Z_I} \frac{1}{q(z)} u_i(z) \pi_{-i}^{\sigma}(z[I]) \pi^{\sigma}(z[I], z). \tag{2.14}$$

([Lanctot et al., 2009, Eq. 6]) Dieser Wert ist im Erwartungswert gleich dem counterfactual value:

$$\mathbb{E}_{j \sim q_j}[\tilde{v}_i(\sigma, I|j)] = v_i(\sigma, I). \tag{2.15}$$

([Lanctot et al., 2009, Lemma 1]) Damit ist $\tilde{v}_i$ ein unverzerrter Schätzer von v_i.

Die Wahl einer Menge Q zusammen mit den Sampling Wahrscheinlichkeiten definiert einen vollständigen sample basierten CFR Algorithmus. Anstatt vollständige Game Tree Traversierungen durchzuführen, sampelt der Algorithmus einen dieser Blöcke und untersucht nur die terminal histories in diesem Block.

2.3.3 Outcome Sampling MCCFR

2.3.4 External Sampling MCCFR

In External Sampling MCCFR werden nur die Aktionen des Gegners und des Zufalls gesampelt, also die Entscheidungen, die extern zum betrachteten Spieler sind. Für jede reine Strategie τ von Gegner und Zufall wird ein Block $Q_\tau \in Q$ definiert. Die Strategie τ ist eine deterministische Abbildung von Informationssets $I \in \mathcal{I}_c \cup \mathcal{I}_{N\setminus\{i\}}$ zu Aktionen $A(I)$.

Die Blockwahrscheinlichkeiten werden basierend auf den Verteilungen f_c und σ_{-i} zugewiesen:

$$q_\tau = \prod_{I \in \mathcal{I}_c} f_c(\tau(I)|I) \prod_{I \in \mathcal{I}_{N\setminus\{i\}}} \sigma_{-i}(\tau(I)|I). \tag{2.16}$$

Der Block Q_τ enthält alle terminal histories z, die mit τ konsistent sind, das heißt, wenn ha ein Präfix von z ist mit $h \in I$ für ein $I \in \mathcal{I}_{-i}$, dann gilt $\tau(I) = a$. In der Praxis wird τ nicht explizit gesampelt, sondern die einzelnen Aktionen werden nur bei Bedarf gesampelt.

Diese Blockwahrscheinlichkeiten ergeben $q(z) = \pi^\sigma_{-i}(z)$. Der Algorithmus iteriert über jeden Spieler $i \in N$ und führt für jeden eine Post-Order Tiefensuche durch den Game Tree durch. Während der Traversierung werden Aktionen an jeder History h gesampelt, für die $P(h) \neq i$ gilt. Diese gesampelten Entscheidungen werden gespeichert, sodass die gleichen Aktionen für alle Histories im selben Informationsset verwendet werden. Aufgrund von perfect recall kann während dieser Traversierung nie mehr als eine History aus dem selben Informationsset besucht werden.

Für jedes besuchte Informationsset werden die sampled counterfactual regrets berechnet und zu den Gesamtregrets hinzugefügt:

$$\tilde{r}(I, a) = (1 - \sigma(a|I)) \sum_{z \in Q \cap Z_I} u_i(z)\pi^\sigma_i(z[I]a, z). \tag{2.17}$$

([Lanctot et al., 2009, Eq. 11]) Die Summation kann während der Traversierung effizient berechnet werden, indem für jede History eine gewichtete Summe der Utilities aller von dieser History erreichbaren terminal histories aufrechterhalten wird.

2.3.5 Theoretische Eigenschaften

Der M Wert

Um die Regret Bounds für MCCFR Varianten zu formulieren, wird ein strukturelles Maß der Spielkomplexität eingeführt: der M Wert. Dieser Wert hängt von der spezifischen Struktur des extensiven Spiels ab und ermöglicht eine präzisere Abschätzung der Konvergenzrate als die einfache Anzahl der Information Sets.

Sei $\vec{a}_i$ eine Teilsequenz einer Historie, die nur die Aktionen von Spieler i in dieser Historie enthält. Sei $\vec{A}_i$ die Menge aller solcher Aktionssequenzen von Spieler i. Für eine

Aktionssequenz $\vec{a}_i$ sei $\mathcal{I}_i(\vec{a}_i)$ die Menge aller Information Sets, bei denen die Aktionssequenz von Spieler i bis zu diesem Information Set genau $\vec{a}_i$ ist. Der M Wert für Spieler i ist dann definiert als

$$M_i = \sum_{\vec{a}_i \in \vec{A}_i} \sqrt{|\mathcal{I}_i(\vec{a}_i)|}. \tag{2.18}$$

([Lanctot et al., 2009])

Der M Wert summiert also über alle möglichen Aktionssequenzen von Spieler i die Wurzel der Anzahl der Information Sets, die zu dieser Sequenz gehören. Dies erfasst, wie sich die Information Sets über die verschiedenen Aktionssequenzen verteilen.

Für den M Wert gilt die folgende Ungleichung:

$$\sqrt{|\mathcal{I}_i|} \leq M_i \leq |\mathcal{I}_i|, \tag{2.19}$$

wobei beide Seiten dieser Schranke von bestimmten Spielen erreicht werden können ([Lanctot et al., 2009]).

Die untere Schranke $\sqrt{|\mathcal{I}_i|}$ wird erreicht, wenn alle Information Sets zu derselben Aktionssequenz gehören. Die obere Schranke $|\mathcal{I}_i|$ wird erreicht, wenn jede Aktionssequenz zu genau einem Information Set führt.

Als Beispiel betrachten wir Kuhn Poker. Beide Spieler haben jeweils sechs Information Sets: $|\mathcal{I}_0| = |\mathcal{I}_1| = 6$.

Für Spieler 0 gibt es zwei Aktionssequenzen mit Information Sets: die leere Sequenz [] mit drei Information Sets $(J, (), 0)$, $(Q, (), 0)$, $(K, (), 0)$ am Spielbeginn, sowie die Sequenz $['check']$ mit drei Information Sets $(J, ('check', 'bet'), 0)$, $(Q, ('check', 'bet'), 0)$, $(K, ('check', 'bet'), 0)$ nach einem Check von Spieler 0 und einem Bet von Spieler 1. Somit gilt $M_0 = \sqrt{3} + \sqrt{3} = 2\sqrt{3} \approx 3.46$.

Für Spieler 1 gibt es nur eine Aktionssequenz mit Information Sets: die leere Sequenz [] mit sechs Information Sets, nämlich drei nach einem Check von Spieler 0 und drei nach einem Bet von Spieler 0. Somit gilt $M_1 = \sqrt{6} \approx 2.45$.

Die Verifikation der Schranken ergibt: $\sqrt{6} \approx 2.45 \leq M_0 = 3.46 \leq 6$ und $\sqrt{6} \approx 2.45 \leq M_1 = 2.45 \leq 6$. Für Spieler 1 wird die untere Schranke erreicht, da alle sechs Information Sets zur selben Aktionssequenz gehören. Für Spieler 0 liegt M_0 zwischen den Schranken, da die Information Sets auf zwei verschiedene Aktionssequenzen verteilt sind.

Der M Wert wird sowohl in den folgenden Regret Bounds für Vanilla CFR als auch für MCCFR verwendet.

Regret Bounds

Mit Hilfe des M Werts können präzisere Regret Bounds für Vanilla CFR formuliert werden.

$$R_i^T \leq \Delta_{u,i} M_i \sqrt{|A_i|} / \sqrt{T}, \tag{2.20}$$

([Lanctot et al., 2009, Thm. 3]) wobei $\Delta_{u,i} = \max_z u_i(z) - \min_z u_i(z)$ der Nutzenbereich für Spieler i ist und $|A_i| = \max_{h:P(h)=i} |A(h)|$ die maximale Anzahl verfügbarer Aktionen für Spieler i bezeichnet.

Diese Schranke verbessert die ursprüngliche Bound aus Theorem 4 des Zinkevich Papers, die $|\mathcal{I}_i|$ anstelle von M_i verwendet. Da $M_i \leq |\mathcal{I}_i|$ gilt, ist die neue Schranke mindestens so gut wie die ursprüngliche und oft deutlich besser. Die Konvergenzrate bleibt bei $O(1/\sqrt{T})$, wobei der konstante Faktor durch die strukturabhängige Komplexität M_i statt der einfachen Anzahl der Information Sets bestimmt wird.

Für die MCCFR Varianten können probabilistische Regret Bounds angegeben werden. External Sampling MCCFR benötigt nur einen konstanten Faktor mehr Iterationen als Vanilla CFR. Für jeden Parameter $p \in (0, 1]$ gilt mit Wahrscheinlichkeit mindestens $1-p$:

$$R_i^T \leq \left(1 + \sqrt{\frac{2}{p}}\right) \Delta_{u,i} M_i \sqrt{|A_i|}/\sqrt{T}. \tag{2.21}$$

([Lanctot et al., 2009, Thm. 4])

Obwohl External Sampling die gleiche Ordnung von Iterationen benötigt wie Vanilla CFR, muss der Algorithmus in jeder Iteration nur einen Bruchteil des Spielbaums traversieren. Für ausgewogene Spiele, in denen die Spieler ungefähr gleich viele Entscheidungen treffen, betragen die Iterationskosten von External Sampling $O(\sqrt{|H|})$, während Vanilla CFR $O(|H|)$ benötigt. Dies bedeutet, dass External Sampling MCCFR asymptotisch weniger Zeit benötigt, um ein approximatives Gleichgewicht zu berechnen als Vanilla CFR ([Lanctot et al., 2009]).

2.4 CFR+

2.4.1 Unterschiede zu Vanilla CFR

CFR+ ist eine Variante von CFR, die deutlich schneller konvergiert als Standard CFR. Cepheus, das erste Programm welches Heads up no Limit Holdem erfolgreich gelöst hat.

Erste Änderung: CFR+ verwendet eine linear gewichtete Durchschnittsstrategie statt der uniformen Durchschnittsstrategie von CFR. Die gewichtete Durchschnittsstrategie ist definiert als $\bar{\sigma}_p^T = 2/(T^2 + T) \sum_{t=1}^{T} t\sigma_p^t$, wobei spätere Iterationen stärker gewichtet werden.

CFR+ verwendet wie Vanilla CFR kein Sampling. Viele andere CFR Implementierungen nutzen Sampling Techniken zur Beschleunigung, während CFR+ und Vanilla CFR den gesamten Spielbaum in jeder Iteration traversieren.

Zweite Änderung: CFR+ führt alternierende Updates durch. Im Gegensatz zum ursprünglichen CFR, das die Regrets beider Spieler simultan aktualisiert, aktualisiert CFR+ die Spieler abwechselnd.

Dritte und wichtigste Änderung: CFR+ verwendet Regret Matching+ anstelle von Regret Matching. Regret Matching+ setzt negative Regrets auf null zurück, statt sie zu ignorieren, was zu besseren Tracking Regret Eigenschaften führt.

2.4.2 Regret Matching+

Regret Matching+ ist ein Regret minimierender Algorithmus, der sehr ähnlich zu Regret Matching operiert. Der entscheidende Unterschied liegt im Umgang mit negativen Regrets: Während Regret Matching Aktionen mit akkumuliertem negativen Regret ignoriert, setzt Regret Matching+ jeden akkumulierten negativen Regret aktiv auf null zurück.

Formal speichert Regret Matching+ nicht die Regrets $R^t(a)$, sondern verfolgt einen regret ähnlichen Wert:

$$Q^t(a) = \left(Q^{t-1}(a) + \Delta R^t(a)\right)^+, \tag{2.22}$$

wobei $x^+ = \max(x, 0)$ den positiven Anteil bezeichnet. Die Strategie basiert auf diesen Werten:

$$\sigma^t(a) = \frac{Q^{t-1}(a)}{\sum_{b \in A} Q^{t-1}(b)}. \tag{2.23}$$

([Tammelin et al., 2015])

Theorem 1 liefert eine Regret Bound für Regret Matching+. Gegeben sei eine Menge von Aktionen A und eine Sequenz von T Wertfunktionen $v^t : A \to \mathbb{R}$. Es existiere eine Schranke L, sodass $|v^t(a) - v^t(b)| \leq L$ für alle t und $a, b \in A$ gilt. Ein Agent, der nach dem Regret Matching+ Algorithmus handelt, hat dann einen Regret von höchstens

$$R^T \leq L\sqrt{|A|T}. \tag{2.24}$$

([Tammelin et al., 2015, Thm. 1])

Diese Schranke ist identisch mit der Regret Bound von Regret Matching. Folglich hat CFR+ die gleiche theoretische Regret Bound wie CFR. In der Praxis konvergiert CFR+ jedoch deutlich schneller als CFR, was teilweise durch die besseren Tracking Regret Eigenschaften von Regret Matching+ erklärt werden kann.

Die empirische Überlegenheit von Regret Matching+ hat eine intuitive Erklärung. Regret Matching+ übertrifft Regret Matching insbesondere dann, wenn sich die beste Aktion plötzlich ändert. Bei Regret Matching muss der Algorithmus warten, bis eine zuvor schlechte Aktion sich beweist, und muss dabei ihr gesamtes akkumuliertes negatives Regret überwinden. Regret Matching+ kann dagegen sofort die neue beste Aktion spielen, da negatives Regret auf null zurückgesetzt wird und nicht erst überwunden werden muss.

2.4.3 Tracking Regret

Tracking Regret ist ein von Herbster und Warmuth eingeführtes Konzept [Herbster and Warmuth, 1998], das die Performance im Nachhinein gegen eine größere Menge von Alternativen betrachtet. Während Standard Regret nur gegen statische Strategien vergleicht, die immer die gleiche Aktion wählen, erlaubt Tracking Regret den Vergleich gegen Strategien, die ihre Aktion höchstens $(k-1)$ Mal ändern können.

Regret Matching hat sehr schlechte Tracking Regret Eigenschaften. Der Algorithmus kann lineares Regret haben, selbst wenn die Vergleichsstrategie nur einmal wechselt

($k = 2$). Im Gegensatz dazu ist Regret Matching+ der erste Regret Matching basierte Algorithmus mit sublinearem Tracking Regret.

Theorem 2 formalisiert diese Eigenschaft. Für alternative Strategien, die bis zu $(k-1)$ Mal wechseln können, hat Regret Matching+ eine Regret Schranke von

$$R^T \leq kL\sqrt{|A|T}. \tag{2.25}$$

([Tammelin et al., 2015, Thm. 2]) Diese Schranke zeigt, dass der Regret linear mit der Anzahl der Wechsel k skaliert, aber sublinear mit der Anzahl der Iterationen T bleibt.

2.4.4 Linear Weighted Average

CFR+ verwendet eine linear gewichtete Durchschnittsstrategie anstelle der uniformen Durchschnittsstrategie von CFR. Die linear gewichtete Durchschnittsstrategie ist definiert als

$$\bar{\sigma}_p^T = \frac{2}{T^2 + T} \sum_{t=1}^{T} t\sigma_p^t, \tag{2.26}$$

wobei spätere Iterationen stärker gewichtet werden.

Theorem 3 zeigt, dass diese Strategie zu einem ε-Nash Equilibrium konvergiert. Für eine Sequenz von T Verhaltensstrategieprofilen σ^t aus CFR+ für T Iterationen in einem extensiven Spiel, wobei $\max |v_p(l) - v_p(l')| \leq L$ für alle Spieler p und terminale Historien l, l' gilt, ist die linear gewichtete Durchschnittsstrategie ein $2(|\mathcal{I}_1|+|\mathcal{I}_2|)L\sqrt{k}/\sqrt{T}$-Nash Equilibrium ([Tammelin et al., 2015, Thm. 3]).

Die Schranke $\varepsilon = 2(|\mathcal{I}_1|+|\mathcal{I}_2|)L\sqrt{k}/\sqrt{T}$ setzt sich aus folgenden Komponenten zusammen: $|\mathcal{I}_1|+|\mathcal{I}_2|$ ist die Summe der Information Sets beider Spieler, L ist der Nutzenbereich als maximaler Unterschied zwischen terminalen Historien, und $k = \max_{I \in \mathcal{I}} |A(I)|$ ist die maximale Anzahl Aktionen in einem Information Set. Die Schranke impliziert eine Konvergenzrate von $O(1/\sqrt{T})$, die asymptotisch identisch mit der Rate von uniform weighting ist. Theorem 3 gilt nicht für CFR, und im Gegensatz zu CFR+, wo linear weighting die Performance verbessert, verschlechtert linear weighting die Performance von CFR. In der Praxis erreicht die aktuelle Strategie in CFR+ oft bereits eine gute Approximation zum Nash Equilibrium, sodass das aggressive Weighting die Konvergenz weiter beschleunigen kann ([Tammelin et al., 2015]).

2.5 Hold'em Games

In diesem Kapitel stelle ich vor, welche Pokervarianten im Rahmen dieser Arbeit mittels CFR gelöst werden sollen. Hierbei erkläre ich zunächst grundlegend wodurch sich die Untergruppierung von Pokerspielen, Hold'em Spiele, auszeichnen und stelle dann einige bekannte Varianten vor. Denn, wenn, man die Grundstruktur von Hold'em Spielen definiert hat, ist es einfach diese Struktur zu erweitern und alle folgenden Poker Hold'em Varianten darzustellen. Anschließend beschreibe ich die Varianten welche in dieser Arbeit gelöst werden und warum ich diese Auswahl getroffen habe.

Ich beschäftige mich in dieser Arbeit ausschließlich mit Heads-Up Limit Versionen von Hold'em Spielen. Heads-Up bedeutet, dass das Spiel nur zwei Teilnehmer beinhaltet. Die Klassifikation Limit bezieht sich darauf, dass die Setzgrößen der Spieler auf vorher festgelegte Beträge beschränkt sind. Diese zwei Einschränkungen reduzieren die Komplexität enorm. Die folgende Definition gilt für jegliche Heads-Up Limit Hold'em Spiele.

Eine Iteration eines Hold'em Spiels nennt man eine Hand. Eine Hand widerum besteht aus mehreren Runden. In der ersten Runde wird eine bestimmte Anzahl an privaten Karten (Hole Cards) an die zwei Teilnehmer verteilt. In *jeder* Runde wird eine bestimmte Anzahl (0 auch möglich), an öffentlichen Karten (Board Cards) aufgedeckt.

Nachdem Karten ausgeteilt oder aufgedeckt wurden, folgt immer eine Setz Runde, in welcher die Spieler abwechselnd Entscheidungen treffen. Ihnen stehen grundlegend 3 Auswahlmöglichkeiten zur Verfügung: "fold, call und raise„. Die Begrifflichkeiten können sich abhängig von der Quelle und dem Spiel etwas unterscheiden.
Wählt ein Spieler die Aktion *fold*, endet die Hand sofort und der andere Spieler gewinnt den Pot. Der Pot ist die Ansammlung aller Einsätze die im Verlauf der Hand gesetzt wurden.
Bei einem *call*, setzt er so viele Chips in den Pot, bis er genauso viele Chips in den Pot eingezahlt hat wie der andere Spieler. Die Anzahl an Chips kann auch 0 betragen. In Pokervarianten welche zu Unterhaltungs und nicht zu Forschungszwecken gespielt werden, nennt man das dann *check*.
Bei der Aktion *raise*, setzt der Spieler zunächst genauso viel Chips in den Pot wie sein Gegner, analog zum *call*. Anschließend setzt er dann zusätzlich noch einen durch die Spielregeln festgelegten Betrag an Chips.

Die Spieler treffen solange abwechselnd Entscheidungen bis entweder ein Spieler folded, in diesem Fall endet die Hand sofort, oder bis ein Spieler called (solange es nicht die erste Aktion der Runde ist). In dem Fall das ein Spieler called, endet nur die Runde und nicht die Hand. Um eine endlose Abfolge aus Raises zu vermeiden, gibt es eine Obergrenze für die Anzahl an Raises nacheinander ausgeführt werden dürfen. Wenn das Setzlimit erreicht ist, bleiben dem Spieler welcher am Zug ist, nur die Optionen Call und Fold. Wenn bis zum Ende der finalen Setzrunde kein Spieler gefolded hat, kommt es zu einem *Showdown*. Bei einem Showdown offenbaren beide Spieler ihre Hole Cards und bilden aus allen zu Verfügung stehenden Karten (Hole + Board Cards) die bestmögliche Pokerhand. Der Spieler mit der stärkeren Hand gewinnt den Pot, welche möglichen Pokerhände es gibt, und welcher Rangordnung diese folgen, hängt von der Variante des Spiels ab. Eine Übersicht über die verschiedenen Pokerhände und deren Bedeutung findet sich unter `https://en.wikipedia.org/wiki/List_of_poker_hands`. [Southey et al., 2005]

Da wir jetzt eine grundlegende Definition eines Heads-Up Limit Hold'em Spiels haben, können wir nun, durch anpassen einzelner Parameter jegliche Varianten definieren. Die relevantesten Aspekte sind: Die Anzahl der Hole und Board Cards, die Anzahl der Setzrunden, dem Setzlimit pro Runde, den im Deck enthaltenden Karten, die möglichen Pokerhände, die festgelegten Setzgrößen und noch einiges mehr.

2.5.1 Kuhn Poker

Die simpelste definierte Pokervariante ist One Card Poker auch Kuhn Poker gennant. Der Name Kuhn Poker stammt durch den Erfinder dieses Spiels Harald Kuhn, welcher es in seinem Paper aus dem Jahr 1951 mit dem Titel "Simplified Two-Person Poker „beschrieb.
Kuhn Poker wird mit einem Kartendeck bestehend aus 3 Karten gespielt. In seinem Paper sind die Karten mit 1,2 und 3 bennant, wobei 1 die schlechteste und 3 die beste Karte ist. Für diese Arbeit wird die Notation J, Q, K verwendet, die den Rängen eines Standard Kartenspiels entspricht.

Zu Beginn zahlen beide Spieler einen Pflichteinsatz in Höhe eines Chips in den Pot, dies wird auch Ante genannt. Beide Spieler erhalten eine private Karte und es werden keine öffentlichen Karten ausgeteilt. Nach dem austeilen der privaten Karte startet die erste und einzige Setzrunde, in welcher die Spieler jeweils einen Chip in den Pot setzen dürfen, womit das Setzlimit bei Eins liegt. Im Falle eines Showdowns, gewinnt die höhere Karte $J < Q < K$.

Kuhn definiert in seinem Paper auch noch andere Versionen seines Spiels indem er das Verhältnis zwischen Ante und Setzgröße anpasst. Der eben beschriebene Fall in welchem der Ante genauso groß ist wie die Setzgröße, wird von ihm Fall 2 genannt. Fall 1 liegt vor, wenn die Setzgröße kleiner als der Ante ist. Fall 3 tritt ein, wenn der Ante kleiner als die Setzgröße ist und die Setzgröße kleiner als das Doppelte des Antes ist. Fall 4 liegt vor, wenn die Setzgröße dem Doppelten des Antes entspricht. [Kuhn, 1951]

2.5.2 Leduc Hold'em

Leduc Hold'em [Southey et al., 2005] ist eine vereinfachte Hold'em Variante für zwei Spieler.

Leduc Hold'em wird mit einem Kartendeck bestehend aus zwei Suits mit drei Karten pro Suit gespielt, insgesamt sechs Karten. Die Karten Notation verwendet Rank und Suit, beispielsweise Js, Jh, Qs, Qh, Ks, Kh.

Das Spiel besteht aus zwei Runden. In der ersten Runde erhält jeder Spieler eine private Karte. In der zweiten Runde wird eine öffentliche Board Karte aufgedeckt.

Zu Beginn zahlen beide Spieler einen Ante von einem Chip in den Pot. Es gibt ein Maximum von zwei Bets pro Runde. Die Setzgrößen betragen 2 Chips in der ersten Runde und 4 Chips in der zweiten Runde.

Die möglichen Pokerhände sind entweder ein Paar oder nur eine High Card. Ein paar gewinnt gegen eine High Card.

2.5.3 Zwölf Karten Variante

Die folgende Pokervariante ist von mir selbst entworfen worden, als eine Zwischenstufe der Komplexität zwischen Leduc Hold'em und Rhode Island Hold'em. Die 12 Karten Variante verwendet ein Deck mit zwölf Karten: J, Q, K und A in drei Suits (s, h, d).

Es gibt drei Runden. In der ersten Runde erhält jeder Spieler eine private Karte. In der zweiten und dritten Runde wird jeweils eine öffentliche Board Karte aufgedeckt.

Es gibt ein Maximum von zwei Bets pro Runde. Die Setzgrößen betragen 2 Chips in der ersten Runde, 4 Chips in der zweiten Runde und 8 Chips in der dritten Runde. Zu Beginn zahlen beide Spieler einen Ante von 1 Chip in den Pot.

Das Spiel endet nach der dritten Runde mit einem Showdown. Die Hand besteht aus der Kombination der privaten Karte des Spielers und den beiden öffentlichen Board Karten. Die Hand Rankings sind: Three of a Kind schlägt ein Paar, ein Paar schlägt High Card.

2.5.4 Rhode Island Hold'em

Rhode Island Hold'em [Shi and Littman, 2001] wurde von Shi und Littman im Jahr 2001 als Testbed für AI Forschung entwickelt.

Rhode Island Hold'em verwendet ein Standard Kartendeck mit 52 Karten. Zu Beginn zahlen beide Spieler einen Ante von 5 Chips in den Pot. Jeder Spieler erhält eine private Karte.

Das Spiel besteht aus drei Runden. Die erste Runde findet nach dem Austeilen der privaten Karten statt. Die Bet Size beträgt 10 Chips und es gibt ein Limit von drei Raises pro Runde. Nach der ersten Runde wird eine öffentliche Board Karte, der Flop, aufgedeckt. Die zweite Runde findet nach dem Flop statt. Die Bet Size beträgt 20 Chips. Nach der zweiten Runde wird eine weitere öffentliche Board Karte, der Turn, aufgedeckt. Die dritte und finale Runde findet nach dem Turn statt. Die Bet Size beträgt ebenfalls 20 Chips.

Beim Showdown bildet jeder Spieler die beste 3 Card Poker Hand aus seiner privaten Karte und den beiden öffentlichen Board Karten. Die Hand Rankings für 3 Card Poker unterscheiden sich von 5 Card Poker. Three of a Kind schlägt Straight und Flush. Straight schlägt Flush. Flush schlägt Pair und Pair schlägt High Card. Bei einem Unentschieden wird der Pot gleichmäßig geteilt.

2.5.5 Limit Texas Hold'em

Texas Hold'em [Southey et al., 2005] ist das häufigste Format für Hold'em.

Texas Hold'em verwendet ein Standard Kartendeck mit 52 Karten, bestehend aus 2 bis Ass in vier Suits.

Das Spiel besteht aus vier Betting Runden. In der ersten Runde erhalten die Spieler jeweils zwei private Karten. In der zweiten Runde, dem Flop, werden drei Board Karten aufgedeckt. In der dritten Runde, dem Turn, wird eine vierte Board Karte aufgedeckt. In der vierten Runde, dem River, wird eine fünfte Board Karte aufgedeckt.

Es gibt ein Maximum von vier Bets pro Runde. Die festgelegten Bet Sizes betragen 10 Chips in den ersten beiden Runden und 20 Chips in den letzten beiden Runden. In diesem Variation beginnt das Spiel nicht mit einem Ante, bei dem beide Spieler die gleiche

Anzahl an Chips in den Pot zahlen, sondern mit einer Struktur die man Blinds nennt. Dabei zahlt der erste Spieler 5 Chips in den Pot und der zweite 10 Chips.

Die möglichen Pokerhände sind Royal Flush, Straight Flush, Four of a Kind, Full House, Flush, Straight, Three of a Kind, Two Pair, One Pair und High Card. Diese Vielfalt an möglichen Händen trägt zur hohen Komplexität des Spiels bei.

Kapitel 3

Analyse

In diesem Kapitel lege ich dar, welche Komponenten Teil dieser Arbeit sind und welche Anforderungen an diese gestellt werden. Zunächst nenne ich die zu lösenden Spielvarianten und begründe, warum diese ausgewählt wurden. Danach stelle ich die Algorithmen vor, welche verwendet werden. Gegen Ende des Kapitels präsentiere ich die Evaluationsmethoden, welche verwendet werden, um festzustellen, ob die errechneten Strategien ein approximatives Nash Gleichgewicht darstellen.

3.1 Spielvarianten

Im Rahmen meines Exposés hatte ich ursprünglich geplant, die Funktionalität von CFR anhand von drei Pokervarianten zu untersuchen: Kuhn Poker, Leduc Hold'em und eine selbst entwickelte Pokervariante mit einem 12 Karten Deck. Ausserdem war geplant, die Funktionalität des Pokerspiels und des CFR Algorithmus gesondert zu implementieren. Dadurch kann das Projekt später ohne große Probleme um weitere Spielvarianten oder andere Algorithmen erweitert werden. Zur Evaluation soll der Best Response Algorithmus sowie der Erwartungswert im Selfplay verwendet werden.

Die Auswahl der Pokervarianten Kuhn Poker und Leduc Holdem wurde getroffen, da diese 2 Spiele die Pokervarianten mit der geringsten Komplexität darstellen und sich deshalb exzellent für Forschungszwecke eignen. Da ich mir im Klaren war, dass ich mit meinen technischen Gegebenheiten nicht in der Lage sein werde eine komplexe Pokervariante wie Limit Holdem zu lösen brauchte ich eine Stufe dazwischen. Zu diesem Zeitpunkt waren mir die Foschungsspiele Rhode Island Holdem und Royal Holdem noch nicht bekannt. Aus diesem Grund habe ich mir eine eigene Pokervariante mit moderater Komplexität ausgedacht welche sich in der Komplexitätsleiter als drittes ansiedelt. Die Varianten Royal Holdem und Rhode Island Holdem besitzen eine weitaus größere Komplexität als meine eigene Kreation. Ich mache mir keine großen Hoffnungen diese Varianten erfolgreich zu lösen, sie werden jedoch trotzdem Implementiert, da der Aufwand sich in Grenzen hält.

3.2 CFR Solver

In dieser Arbeit werden drei verschiedene Varianten von CFR Algorithmen implementiert: Vanilla CFR, CFR+ und MCCFR mit External Sampling. Vanilla CFR dient als Baseline für den Vergleich aller Varianten und ermöglicht es, die Verbesserungen der anderen Algorithmen zu quantifizieren. MCCFR wurde ausgewählt, um eine Variante zu untersuchen, die nicht den gesamten Game Tree explizit besucht, sondern mittels Sampling arbeitet. CFR+ wurde implementiert, da es die Grundlage für die erfolgreiche Poker KI Libratus bildet, welche in den letzten Jahren bedeutende Erfolge in diesem Bereich erzielt hat.

3.3 Evaluationsmethoden

Zur Evaluation der berechneten Strategien werden zwei Hauptmethoden verwendet. Exploitability Tests mit einem Best Response Agent und die Berechnung des Erwartungswerts im Selfplay.

Der Exploitability Test mit einem Best Response Agent misst, wie stark eine Strategie ausgenutzt werden kann. Ein Best Response Agent spielt optimal gegen die zu evaluierende Strategie. Die Exploitability ist definiert als der erwartete Gewinn des Best Response Agents minus dem erwarteten Gewinn bei einem Nash Gleichgewicht. Eine niedrige Exploitability deutet darauf hin, dass die Strategie nahe an einem Nash Gleichgewicht liegt.

Die Berechnung des Erwartungswerts im Selfplay ermöglicht es, die Performance der Strategie zu bewerten, wenn beide Spieler die gleiche Strategie verwenden. Dies gibt Aufschluss über die Qualität der Strategie in einem symmetrischen Setting.

Zusätzlich werden Konvergenzplots erstellt, um die Entwicklung der Strategiequalität über die Iterationen hinweg zu visualisieren. Diese werden erstellt, indem in regelmäßigen Abständen, die Exploitability mittels des Best Response Agents ermittelt wird.

In diesem Kapitel wurden die Komponenten dieser Arbeit vorgestellt und die Anforderungen an diese definiert. Es wurden die zu untersuchenden Spielvarianten, die zu implementierenden CFR Algorithmen und die verwendeten Evaluationsmethoden dargelegt. Im folgenden Kapitel beschreibe ich wie die Architektur der Software aussieht.

Kapitel 4

Design und Architektur

In diesem Kapitel wird die Architektur des Softwareprojekts beschrieben. Es werden die Designentscheidungen erläutert und die Struktur der einzelnen Komponenten anhand von UML Diagrammen dargestellt.

Ein zentrales Designprinzip des Codes ist die Trennung der Game Engine vom CFR Algorithmus. Die Game Engine wird als separate Entität implementiert, die die Spiellogik und Spielzustandsverwaltung übernimmt. Der CFR Algorithmus nutzt die öffentlichen Methoden der Game Engine, ohne direkten Zugriff auf die interne Implementierung der Spiellogik zu haben.

Diese Architektur ermöglicht es, das Projekt später ohne große Probleme um weitere Spielvarianten oder andere Algorithmen zu erweitern. Neue Pokervarianten können implementiert werden, ohne den CFR Algorithmus zu ändern, und neue Algorithmen können hinzugefügt werden, ohne die Game Engine zu modifizieren. Diese Trennung von Algorithmus und Spiellogik erhöht die Wartbarkeit und Erweiterbarkeit des Systems.

4.1 Game Environment

In diesem Kapitel wird der Aufbau der Spielumgebung mithilfe von Klassendiagrammen beschrieben. Die Spielumgebung ist für die Logik des Pokerspiels zuständig, sowie dazu den Spielzustand zu verwalten.

Eine Spielumgebung ist unterteilt in die Klassen Player(2 Teilnehmer des Spiels), Dealer(Verwaltet Deck), Judger(bestimmt den Gewinner) und Game(Verwendet andere Komponenten). Dies wird bei allen Varianten ausser Kuhn Poker zusätzlich noch durch eine Round Klasse erweitert, welche den Übergang zwischen Setzrunden steuert. Diese Architektur wurde von RLCard übernommen.

Abbildung 4.1 zeigt das Klassendiagramm der Game Environment für verschiedene Pokervarianten. Im Diagramm ist zu erkennen, dass die Game Environments voneinander erben, wodurch einige Bestandteile wiederverwendet werden können.

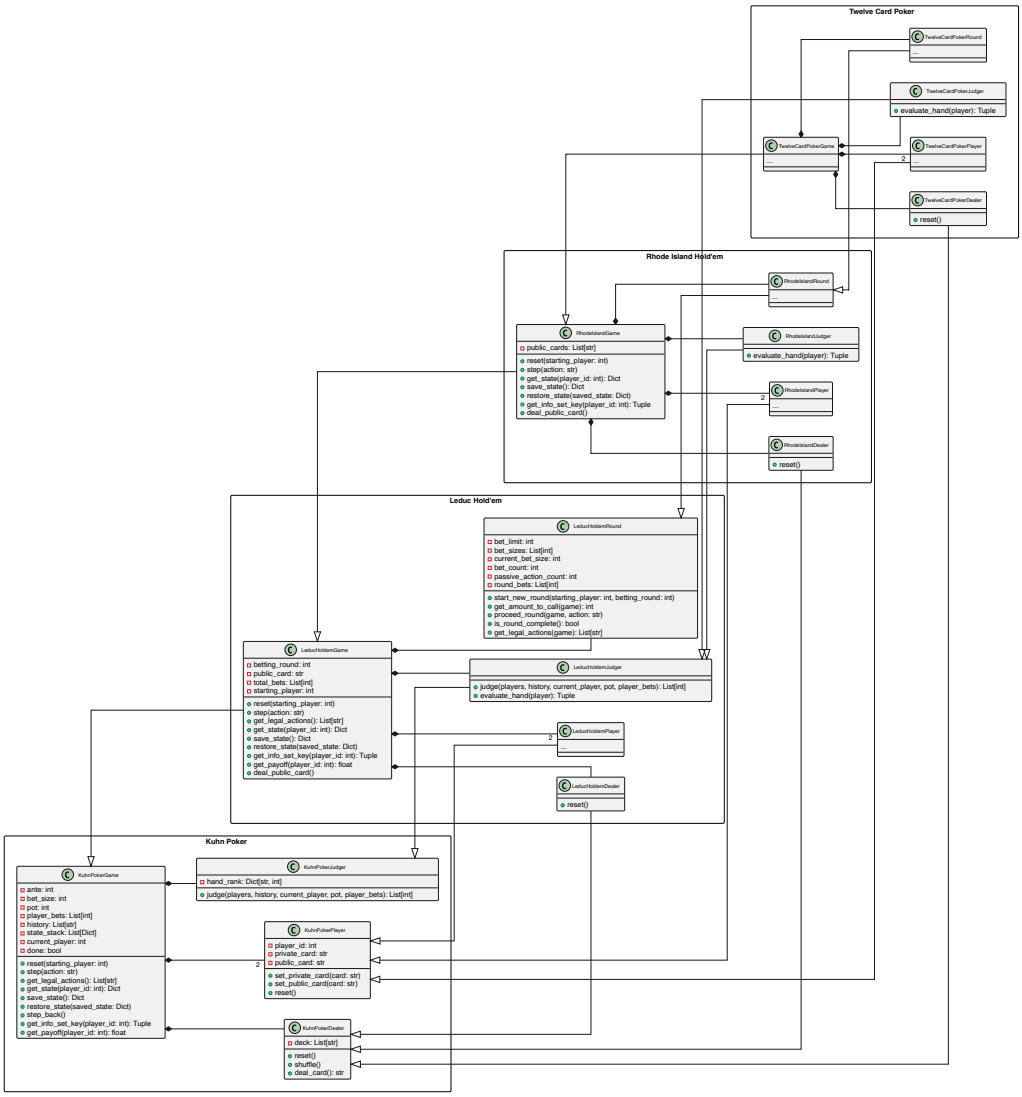

Abbildung 4.1: Klassendiagramm der Game Environment für verschiedene Pokervarianten

4.2 CFR Solver

Abbildung 4.2 zeigt das Klassendiagramm der CFR Solver Architektur.

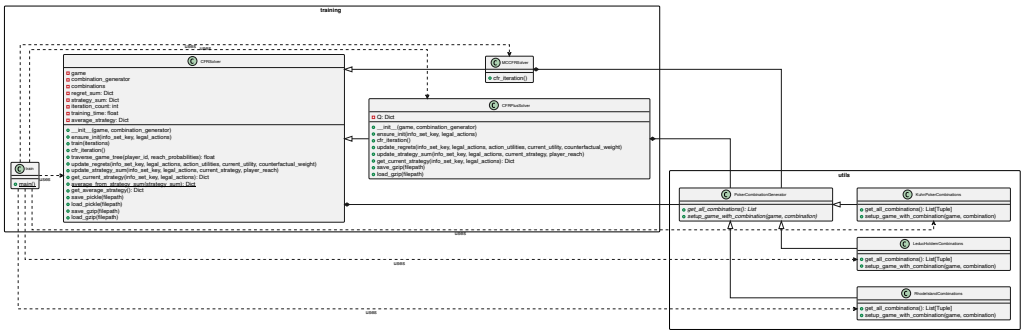

Abbildung 4.2: Klassendiagramm der CFR Solver Architektur

Das Diagramm zeigt die drei implementierten Solver Varianten CFRSolver, CFRPlusSolver und MCCFRSolver, wobei CFRPlusSolver und MCCFRSolver von CFRSolver erben.

4.2.1 CFR Training Ablauf

Abbildung 4.3 zeigt den Ablauf des CFR Trainings als Aktivitätsdiagramm.

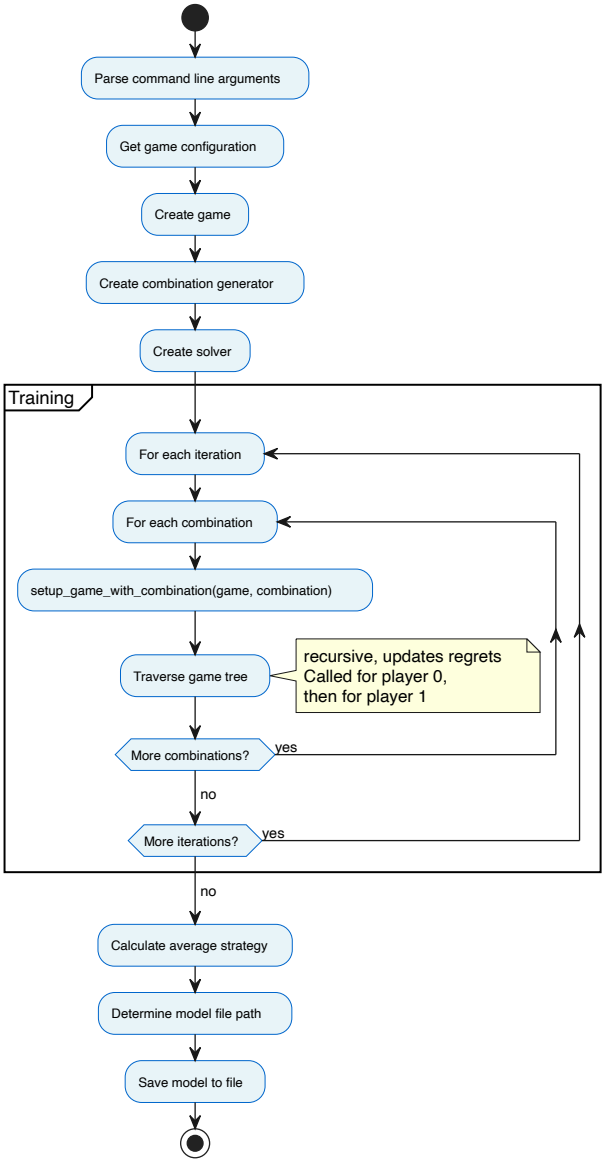

Abbildung 4.3: Aktivitätsdiagramm des CFR Trainings

Der Prozess beginnt mit dem Einlesen der Kommandozeilenargumente. Anschließend wird die Spielkonfiguration geladen. Danach werden das Spiel, der Combination Generator und der Solver erstellt. Der Combination Generator erzeugt alle möglichen Kartenkombinationen. Das Training besteht aus zwei verschachtelten Schleifen: einer äußeren Schleife über die Iterationen und einer inneren Schleife über alle Kombinationen. Für jede Kombination wird das Spiel initialisiert, der Spielbaum durchlaufen und die Regrets aktualisiert. Nach Abschluss aller Iterationen wird die durchschnittliche Strategie berechnet und das trainierte Modell gespeichert.

4.2.2 Traverse Game Tree

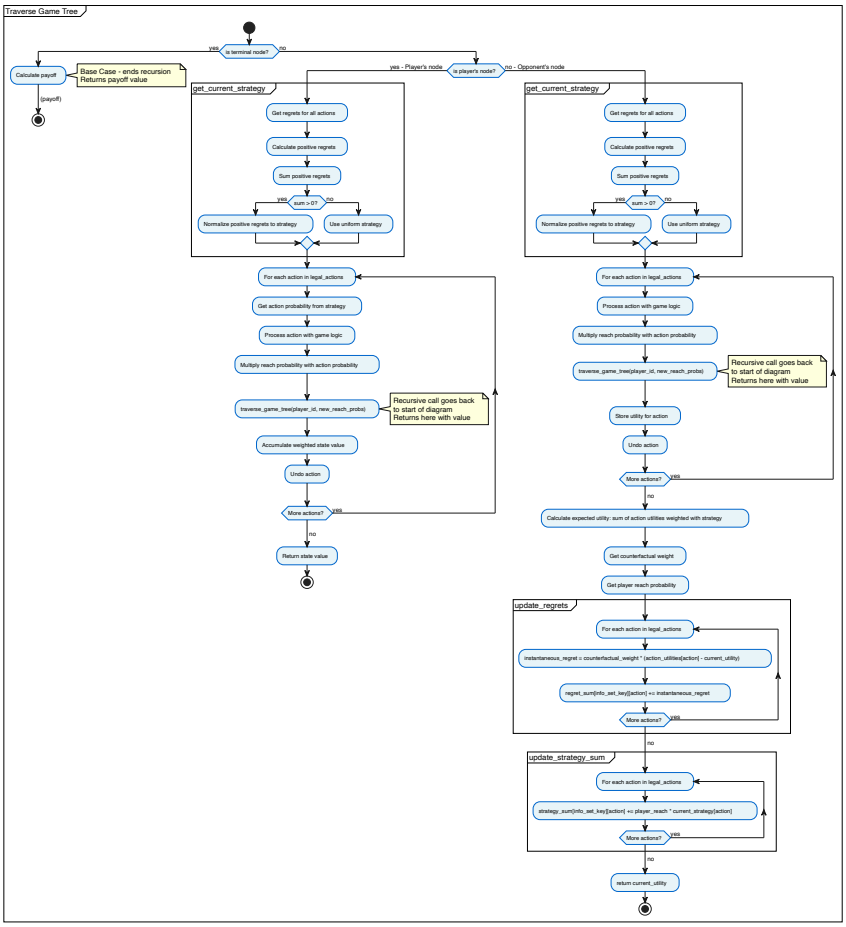

Abbildung 4.4: Aktivitätsdiagramm der Traversierung des Spielbaums

Abbildung 4.4 zeigt den Ablauf der rekursiven Traversierung des Spielbaums. Der Algorithmus beginnt mit einer Prüfung, ob es sich um einen Terminalknoten handelt. Falls ja, wird der Payoff berechnet und zurückgegeben. Handelt es sich um einen Spielerknoten, wird die aktuelle Strategie mittels Regret Matching bestimmt. Für jede legale Aktion wird die Aktion ausgeführt, die Reach Wahrscheinlichkeiten aktualisiert und rekursiv der Spielbaum durchlaufen. Nach dem rekursiven Aufruf wird der gewichtete Zustandswert akkumuliert und die Aktion rückgängig gemacht. Handelt es sich um einen Gegnerknoten, wird die Gegnerstrategie verwendet, um die Aktionswahrscheinlichkeiten zu bestimmen und den Spielbaum entsprechend zu traversieren.

4.2.3 Strategy Agent und Logger

Nach dem Training des CFR Solvers wird die errechnete durchschnittliche Strategie von einem Strategy Agent ausgeführt. Der Strategy Agent bestimmt das entsprechende Informationsset für den aktuellen Spielzustand und ermittelt die zugehörigen Aktionswahrscheinlichkeiten aus der Strategie. Anschließend wählt er basierend auf dieser Wahrscheinlichkeitsverteilung zufällig eine Aktion aus und führt sie aus.

Kapitel 5

Implementierung

5.1 Game Environment

Grundstruktur Player, Judger, Dealer, Game, Round

5.2 CFR Solver

Initialisierung, Combination Generator

Training Loop

Regret Matching

Average Strategy

Storage

5.3 Evaluationsmethoden

Best Response Agent

Exploitability Berechnung

Selfplay

Kapitel 6

Evaluation

6.1 Ergebnisse

Kuhn Poker Ergebnisse

Leduc Hold'em Ergebnisse

Eigene Version

Rhode Island Hold'em Ergebnisse

6.2 Vergleich der Algorithmen

CFR vs. CFR+

CFR vs. MCCFR

Konvergenzplots

Abstraktionsmethoden, Multithread

6.3 Vergleich mit Literaturwerten und anderen Modellen

Kuhn 1951 (analytische Lösung)

Zinkevich 2007 (CFR)

Lanctot et al. 2009 (MCCFR)

Tammelin et al. 2015 (CFR+)

OpenSpiel/RL Card

Random Leute in Foren

Kapitel 7

Fazit

7.1 Zusammenfassung

Ergebnisse

Erkenntnisse

7.2 Lessons Learned

Herausforderungen

Lösungsansätze

7.3 Future Work

Literaturverzeichnis

[Herbster and Warmuth, 1998] Herbster, M. and Warmuth, M. K. (1998). Tracking the best expert. *Machine Learning*, 32(2):151–178.

[Kuhn, 1951] Kuhn, H. W. (1951). Simplified two-person poker. *Contributions to the Theory of Games*, 1:97–103.

[Lanctot et al., 2009] Lanctot, M., Waugh, K., Zinkevich, M., and Bowling, M. (2009). Monte carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems*, pages 1078–1086.

[Osborne and Rubinstein, 1994] Osborne, M. J. and Rubinstein, A. (1994). *A Course in Game Theory*. MIT Press, Cambridge, Massachusetts.

[Shi and Littman, 2001] Shi, J. and Littman, M. L. (2001). Rhode island hold'em poker. In *AAAI/IAAI*, pages 602–607.

[Southey et al., 2005] Southey, F., Bowling, M., Larson, B., Piccione, C., Burch, N., Billings, D., and Rayner, C. (2005). Bayes' bluff: Opponent modelling in poker. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence (UAI 2005)*.

[Tammelin et al., 2015] Tammelin, O., Burch, N., Johanson, M., and Bowling, M. (2015). Solving heads-up limit texas hold'em. In *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence (IJCAI 2015)*, pages 645–652.

[Zinkevich et al., 2007] Zinkevich, M., Johanson, M., Bowling, M., and Piccione, C. (2007). Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*.